Machine Learning for Molecular Engineering
3/7/10/20.01 (U) 3/7/10/20.C51 (G)
Spring 2025

Problem Set #6

Date: April 10, 2025
Due: May 12, 2025 @ 5 pm

Instructions

- **Important:** This problem set requires a GPU. Before you start in Google Colab, find `Notebook Settings` under the `Edit` menu. In the `Hardware accelerator` drop-down, select a GPU as your hardware accelerator.
- Please submit your solution on Canvas as an IPython (Jupyter) Notebook file, `*.ipynb`. We highly suggest using Google Colab for this course and using this template for the second problem set. If you have not used Google Colab, you might find this example notebook helpful. After opening this template link, you should select "Save a copy in Drive" from the File menu. When you've completed the assignment, you can download the `*.ipynb` file from the File menu to upload to Canvas. Your submission should contain *all* of the code that is needed to fully answer the questions in this problem set when executed from top to bottom and should run without errors (unless intentional). Please ensure that your plots are visible in the output file and that you are printing any additional comments or variables as needed. Commenting your code in-line and adding any additional comments in markdown (as "Text cells") will help the grader understand your approach and award partial credit.
- Note that collaboration between students is encouraged, however every student must be responsible for all the work in their individual submission. I.e., discussing solutions and debugging together should be considered fruitful teamwork, however physically taking the keyboard of another student and directly entering code for their submission would be intellectually dishonest and as such is prohibited. You should first read through and attempt every problem before discussing with others. Please list the names of all collaborators in your .ipynb submission file.
- Template for this PSET is available here.

Background

Protein Language Models (pLMs), including Meta's ESMFold, are used to generate protein structures given an input sequence. ESMFold stems from the ESM2 model, which generates embeddings for protein sequences, leveraging the transformer architecture (similar to the Bidirectional Encoder Representations from Transformers (BERT) model in the context of Natural Language Processing (NLP)).

ESM2 was trained using a masked language modeling objective, where a given position in the input amino acid sequence is masked and the model has to predict the most likely amino acid at that position. This is repeated for every amino acid in the sequence. The transformer architecture helps

capture long-range relationships between different parts of a protein sequence without explicitly providing any information about the protein's 3D structure. ESM2 was trained on approximately 65 million sequences from the UniRef protein sequence database. The ESM2 model you will be working with in this PSET has around 3 billion parameters, 36 layers, 40 attention heads, and a hidden space dimension of 2560.

In part 1, you will leverage a pre-trained ESM2 model to generate embeddings for protein sequences and use them to train a small MLP to predict GO (Gene Ontology) terms for proteins. The GO knowledge base includes subontologies describing molecular functions of proteins, biological processes in which proteins are involved, and cellular components in which they are active. As expected, GO annotations can generally be propagated to homologous proteins. The UniProtKB/Swiss-Prot database contains manually curated GO annotations for thousands of organisms and more than 550,000 proteins. You can learn more about GO terms here.

Here, you will compare the performance of the MLP-ESM2 model with an MLP trained on one-hot encoded sequence data. The first task has been accomplished in recent publication [1]. You can skim these papers if you would like, but that is not required to answer the questions in the PSET. All the required background is explained below.

In part 3, you will use ESMFold to predict the structure of one protein in the validation set.

Part 1: Leverage foundation models to predict protein structure and function

This part will require you to implement a deep neural network in PyTorch. If you need to, please go through the PyTorch tutorial here and the quickstart guide here. You'll want a GPU for this part - request one now.

Part 1.1 (15 points) Generate and Visualize ESM2 Embeddings

The output of layers in ESM models — so-called hidden states — can be converted into vector embeddings for amino acid sequences. These hidden states can be passed through a language modeling head to produce logits at each position, which are then transformed into probabilities using a softmax function. These probabilities can provide interpretable insights into the model's understanding of protein sequences. Extract the ESM2 embeddings for both the training and validation datasets.

Project the ESM2 embeddings to 2 dimensions and visualize them using UMAP. **(2.5 points)**

- What do you observe in terms of the information captured by the ESM2 embeddings for the training vs. validation sets? **(2.5 points)**
- How might the observed patterns in the distribution of training vs. validation data affect out-of-distribution testing and generalization performance? **(5 points)**
- What information about proteins can be useful when doing a train-test-validation split on sequence data? **(5 points)**

Part 1.2 (5 points) Extract and binarize GO terms for training and validation data

Binarize the labels using `MultiLabelBinarizer`, and print the total number of unique GO terms in the dataset. Then, complete the preprocessing of the GO terms for the validation data using the `MultiLabelBinarizer.transform` function.

Part 1.3 (5 points) Creating datasets for model training and validation

Complete the definition of the `ProteinDataset` class shown below. Both the ESM2 embeddings and labels should be converted to `torch.float32`. **(2.5 points)** Then, create datasets and dataloaders for the training and validation data using the `ProteinDataset` class you defined. Don't forget to shuffle the training dataloader but not the validation dataloader. **(2.5 points)**

Part 1.4 (5 points) Define the MLP

Complete the definition of the `ESM2MLP` class with the right dimensions.

Part 1.5 (30 points) Train and Evaluate Model

To train the model, start by setting up the Adam optimizer, with a learning rate `lr` of `1e-4`. (**5 points**) Next, write your training loop using the provided `train_epoch()` function, and train the model for 10 epochs. This is a classification problem, so you will want to compute the binary cross-entropy loss using `nn.BCEWithLogitsLoss()`, which requires you to input classification probabilities and the ground truth labels from your data. (**10 points**) Record the loss across epochs and plot the training loss across epochs. Comment briefly on the trend you observe. (**5 points**)

To evaluate the model, run the provided code cells and answer the following questions:

- Why do we need the sigmoid activation when looking at the predictions and converting them to probabilities? (**5 points**)
- Compute Precision, Recall, and F1-Score using the relevant scikit-learn function and comment on the values. Note the `'average'` argument. (**5 points**)

Part 1.6 (30 points) Compare with MLP trained on one-hot encoded sequences

We will compare the performance of an **MLP trained on ESM2 embeddings** with a simpler **MLP trained on one-hot encoded sequence data** using the validation data. Start by one-hot encoding the sequence data using the `one_hot_encode_sequence` function that you will have to complete (**5 points**). Then, complete the `ProteinOneHotDataset` class and create the training and validation dataloaders (**5 points**). Make sure to use the `collate_fn` to handle variable-length sequences when defining the DataLoader. Implement the small MLP with the architecture specified below (**5 points**). Start by defining the `__init__` function of the `OneHotMLP` class with `hidden_dim = 256`. You will also need to define `input_dim` and `num_classes`.

The class should inherit from `nn.Module`, and use these layers to build the network:

- `nn.Linear`
- `nn.ReLU`

- `nn.Dropout`
- `nn.Sequential`

The feedforward neural network should:

1. Take an input of shape `(batch_size, input_dim)`.
2. Have a hidden layer with **256** nodes and ReLU activation.
3. Map the output to **num_classes** labels.
4. Apply a **0.1** dropout rate after ReLU.

The `forward` method is already implemented for you.

Then, train the model using the same hyperparameters as defined for the MLP trained on ESM2 embeddings (**5 points**).

Evaluate the model on the validation data. Use a threshold of 0.5 to compute the binary predictions (**5 points**).

Compute Precision, Recall, and F1-Score as you did above. Comment on the values and compare performances of the MLP trained on one-hot encodings vs. MLP trained on ESM2 embeddings, and explain potential reasons for differences in performance (**2.5 points**). In your answer, make sure to comment on potential issues with one-hot encodings as well as on the information content of ESM2 embeddings (**2.5 points**).

Part 2: Investigating a protein of interest: Mouse hypothalamic galanin-like neuropeptide (`GALP_MOUSE`) (5 points)

Compare the GO terms predicted by the MLP and the naive approach for `GALP_MOUSE` in the validation set and its experimental annotations. Note that you can ignore the '`|IEA,` ' and '`|IDA`' suffixes in the GO terms. These relate to how the annotations were made - more details can be found here. Use this link to look up the GO terms.

Part 3: Predicting Protein Structure using ESMFold (5 points)

Use the relevant sequence from the validation dataframe to predict the 3D structure `GALP_MOUSE` using ESMFold. Paste the relevant protein sequence and hit 'Fold Sequence'. Please upload below a screenshot of the top predicted structure using the files tab to the left. Include the sequence you pasted into ESMFold in your answer below and comment on the confidence of the top prediction.

References

[1] Zhang, L., Smith, J. & Doe, J. Deep learning models for protein structure prediction. *Nature Machine Intelligence* **6**, 345–356 (2024).